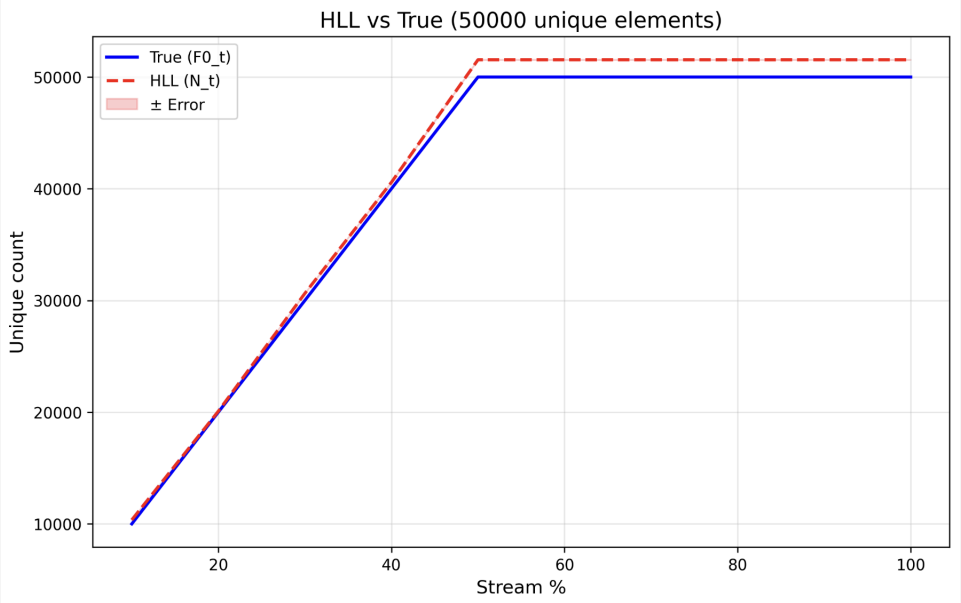
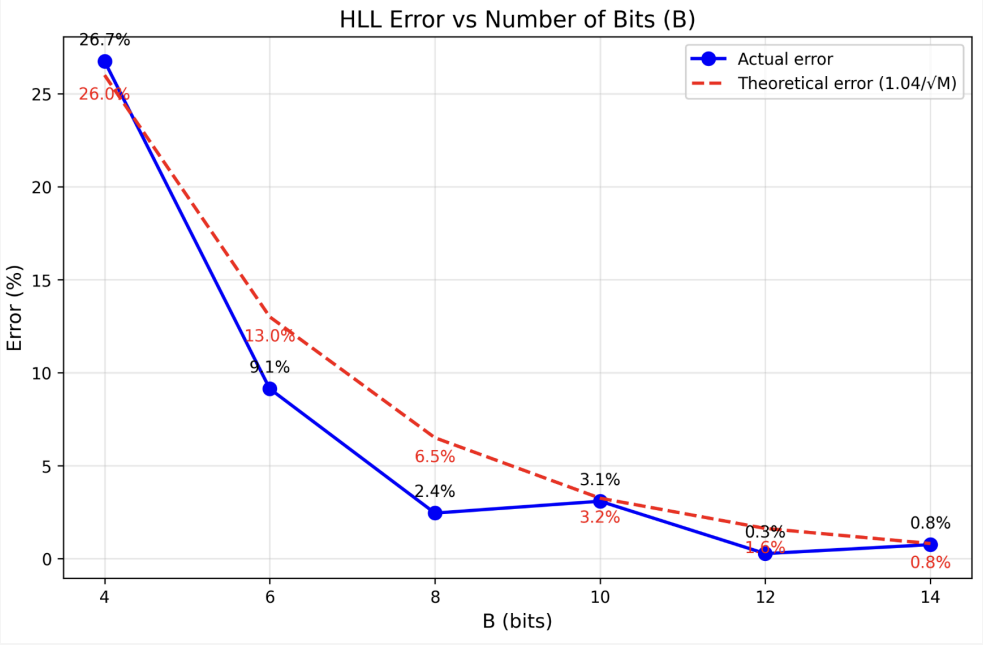
```
// Осина Дарья БПИ-245
```

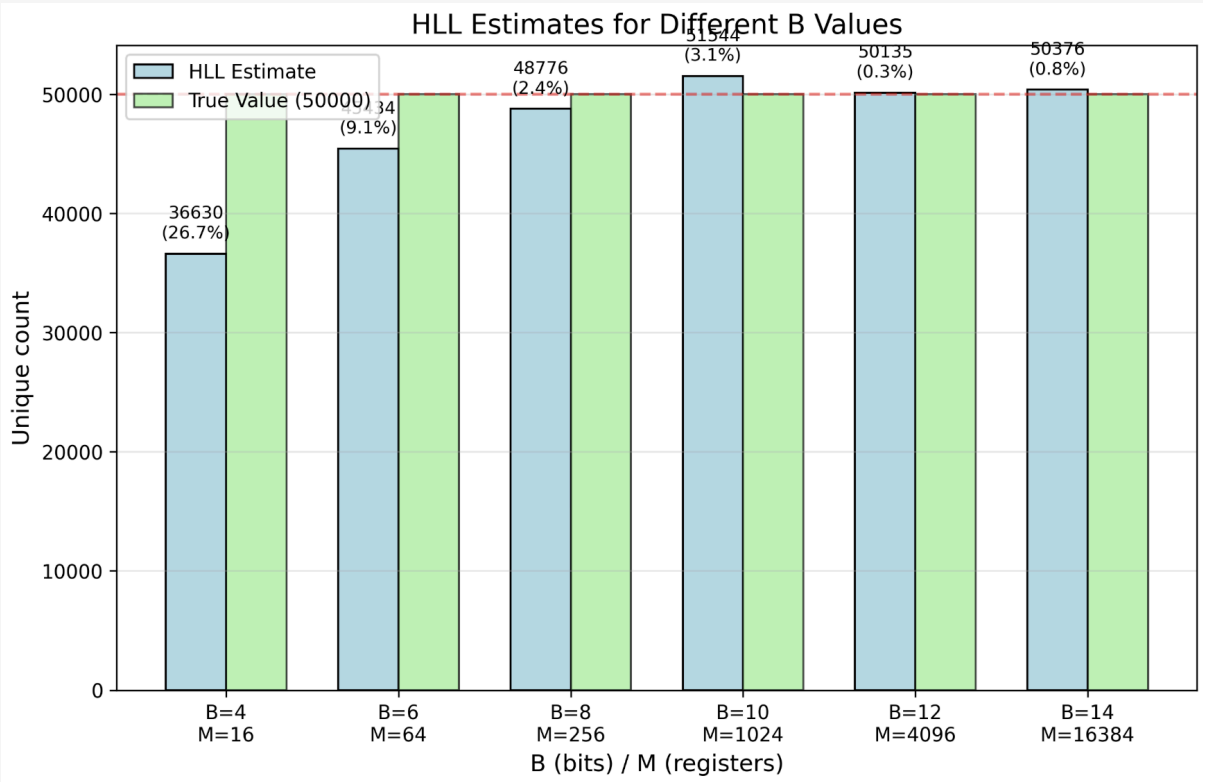
Output: 2945 ms | 16.0 M

```
Анализ:
b=4, M=16, Реально_уникальных=50000, Оценка_моя=36629.6, Относительная_ошибка=26.7409 Ошибка_по_формуле=26%
b=6, M=64, Реально_уникальных=50000, Оценка_моя=45433.5, Относительная_ошибка=9.13309 Ошибка_по_формуле=13%
b=8, M=256, Реально_уникальных=50000, Оценка_моя=48775.6, Относительная_ошибка=2.44886 Ошибка_по_формуле=6.5%
b=10, M=1024, Реально_уникальных=50000, Оценка_моя=51543.7, Относительная_ошибка=3.08742 Ошибка_по_формуле=3.25%
b=12, M=4096, Реально_уникальных=50000, Оценка_моя=50135.3, Относительная_ошибка=0.27069 Ошибка_по_формуле=1.625%
b=14, M=16384, Реально_уникальных=50000, Оценка_моя=50376.5, Относительная_ошибка=0.753047 Ошибка_по_формуле=0.8125%

Дисперсия (b=10):
Средняя_ошибка: 2.32058%
Отклонение: 1.7629%
По формуле: 3.25%
```








```cpp
#include <vector>
#include <cmath>
#include <algorithm>
#include <functional>
#include <iostream>
#include <unordered_set>
#include <string>
#include <fstream>
#include <random>

typedef unsigned int uint;

class RandomStreamGen {
public:
   std::mt19937 rnd;
   std::uniform_int_distribution<int> ch;
   std::uniform_int_distribution<int> len;
   std::string alphabet =
"abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789-";

   RandomStreamGen(int x = 13) :
       rnd(x),
       ch(0, 62),
       len(5, 30) {
   }
```

```
std::string generate_str() {
    int l = len(rnd);
    std::string res;
    res.reserve(l);

    for (int i = 0; i < l; ++i) {
        res += alphabet[ch(rnd)];
    }

    return res;
}

std::vector<std::string> generate(size_t n) {
    std::vector<std::string> s;
    s.reserve(n);

    for (size_t i = 0; i < n; ++i) {
        s.push_back(generate_str());
    }

    return s;
}

std::vector<std::string> gen_with_uniq(size_t n, size_t nUniq) {
    std::vector<std::string> uniqStr;
    std::unordered_set<std::string> uniqSet;
    std::vector<std::string> stream;
    stream.reserve(n);

    while (uniqStr.size() < nUniq) {
        std::string str = generate_str();
        if (uniqSet.insert(str).second) {
            uniqStr.push_back(str);
        }
    }

    stream.insert(stream.end(), uniqStr.begin(), uniqStr.end());
    std::uniform_int_distribution<size_t> v(0, nUniq - 1);

    while (stream.size() < n) {
        size_t i = v(rnd);
        stream.push_back(uniqStr[i]);
    }
```

```
        return stream;
    }

    std::vector<std::vector<std::string>> split(std::vector<std::string>& s,
std::vector<double>& percent) {
        std::vector<std::vector<std::string>> v;
        v.reserve(percent.size());

        for (double perc : percent) {
            size_t end = static_cast<size_t>(s.size() * perc / 100.0);
            std::vector<std::string> part(s.begin(), s.begin() + end);
            v.push_back(part);
        }
        return v;
    }
};

class HashFuncGen {
public:
    uint seed;

    HashFuncGen(uint s = 13) : seed(s) {}

    uint murmur_hash(const std::string& s) const {
        uint c1 = 0xcc9e2d51;
        uint c2 = 0x1b873593;
        uint r1 = 15;
        uint r2 = 13;
        uint m = 5;
        uint n = 0xe6546b64;
        uint h = seed;
        size_t l = s.length();

        for (size_t i = 0; i < l; i += 4) {
            uint k = 0;
            for (int j = 0; j < 4 && (i + j) < l; ++j) {
                k |= (static_cast<uint>(s[i + j]) << (8 * j));
            }
            k *= c1;
            k = (k << r1) | (k >> (32 - r1));
            k *= c2;
            h ^= k;
            h = (h << r2) | (h >> (32 - r2));
            h = h * m + n;
        }
```

```
        h ^= l;
        h ^= h >> 16;
        h *= 0x85ebca6b;
        h ^= h >> 13;
        h *= 0xc2b2ae35;
        h ^= h >> 16;
        return h;
    }
};

class HyperLogLog {
public:
    int b;
    int m;
    double alph;
    std::vector<uint> reg;
    std::function<uint(const std::string&)> foo;

    int cnt_zeros(uint x, int maxb) {
        if (x == 0) return maxb;
        int count = 0;
        while (count < maxb && ((x >> (maxb - 1 - count)) & 1) == 0) {
            ++count;
        }
        return count + 1;
    }

    int get_m() {
        return m;
    }

    int get_b() {
        return b;
    }

    const std::vector<uint>& get_registers() {
        return reg;
    }

    HyperLogLog(int bits = 8, std::function<uint(const std::string&)> hf = nullptr)
: b(bits) {
        m = 1 << b;
        reg.resize(m, 0);
```

```
        if (m == 16) {
            alph = 0.673;
        }
        else if (m == 32) {
            alph = 0.697;
        }
        else if (m == 64) {
            alph = 0.709;
        }
        else {
            alph = 0.7213 / (1.0 + 1.079 / m);
        }

        if (!hf) {
            HashFuncGen h;
            foo = [h](const std::string& s) { return h.murmur_hash(s); };
        } else {
            foo = hf;
        }
    }

    void add(const std::string& s) {
        uint el = foo(s);
        uint i = el >> (32 - b);
        uint val = el & ((1u << (32 - b)) - 1);
        int zeroKol = cnt_zeros(val, 32 - b);

        if (zeroKol > reg[i]) {
            reg[i] = zeroKol;
        }
    }

    double e() {
        double sum = 0.0;
        for (int i = 0; i < m; ++i) {
            sum += 1.0 / (1 << reg[i]);
        }

        double e = alph * m * m / sum;

        if (e <= 2.5 * m) {
            int z = std::count(reg.begin(), reg.end(), 0);
            if (z) {
                e = m * log(m / (double)z);
            }
```

```
        }
        if (e > (1.0 / 30.0) * (1ULL << 32)) {
            e = -(1ULL << 32) * log(1.0 - e / (1ULL << 32));
        }

        return e;
    }

    void clear() {
        std::fill(reg.begin(), reg.end(), 0);
    }

    static size_t cnt_unique(const std::vector<std::string>& data) {
        std::unordered_set<std::string> s(data.begin(), data.end());
        return s.size();
    }

    double err_small() {
        return 1.04 / sqrt(m);
    }

    double err_big() {
        return 1.3 / sqrt(m);
    }
};

class Analyzer {
private:
    RandomStreamGen gen;

public:
    void test() {
        std::cout << "Анализ:\n";
        std::vector<int> v = {4, 6, 8, 10, 12, 14};
        auto stream = gen.gen_with_uniq(100000, 50000);
        double exact = HyperLogLog::cnt_unique(stream);

        for (int b : v) {
            HyperLogLog hll(b);
            for (auto& s : stream) {
                hll.add(s);
            }
            double est = hll.e();
            double err = fabs(est - exact) / exact * 100;
            double th_err = 1.04 / sqrt(1 << b) * 100;
```

```
            std::cout << "b=" << b << ", M=" << (1 << b) << ", Реально_уникальных="
<< exact << ", Оценка_моя=" << est << ", Относительная_ошибка=" << err << "
Ошибка_по_формуле=" << th_err << "%\n";
        }
    }

    void test_D(int b = 10, int maxb = 100) {
        std::cout << "\nДисперсия (b=" << b << "):\n";
        std::vector<double> errs;
        double exact = 25000;

        for (int t = 0; t < maxb; ++t) {
            auto stream = gen.gen_with_uniq(50000, static_cast<size_t>(exact));
            HyperLogLog hll(b);
            for (auto& s : stream) hll.add(s);

            double err = fabs(hll.e() - exact) / exact * 100;
            errs.push_back(err);
        }

        double r1 = 0, var = 0;
        for (double e : errs) r1 += e;
        r1 /= maxb;
        for (double e : errs) var += (e - r1) * (e - r1);
        var /= maxb;
        double r2 = sqrt(var);
        double r3 = 1.04 / sqrt(1 << b) * 100;

        std::cout << "Средняя_ошибка: " << r1 << "%\n";
        std::cout << "Отклонение: " << r2 << "%\n";
        std::cout << "По формуле: " << r3 << "%\n";
    }
};

int main() {
    Analyzer a;
    a.test();
    a.test_D(10, 100);
    return 0;
}
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

unq = 50000
arr = {
   'b': [4, 6, 8, 10, 12, 14],
   'actual_error': [26.7409, 9.13309, 2.44886, 3.08742, 0.27069, 0.753047],
   'theoretical_error': [26.0, 13.0, 6.5, 3.25, 1.625, 0.8125],
   'M': [16, 64, 256, 1024, 4096, 16384],
   'estimate': [36629.6, 45433.5, 48775.6, 51543.7, 50135.3, 50376.5]
}

def gr1():
   df = pd.read_csv('res1.csv')
   plt.figure(figsize=(10,6))
   plt.plot(df['pct'], df['exact'], 'b-', linewidth=2, label='True (F0_t)')
   plt.plot(df['pct'], df['est'], 'r--', linewidth=2, label='HLL (N_t)')
   plt.fill_between(df['pct'], df['est']-df['err'], df['est']+df['err'],
                    alpha=0.2, color='r', label='± Error')
   plt.xlabel('Stream %', fontsize=12)
   plt.ylabel('Unique count', fontsize=12)
   plt.title('HLL vs True (50000 unique elements)', fontsize=14)
   plt.legend(loc='best')
   plt.grid(True, alpha=0.3)
   plt.savefig('plot1_hll_vs_true.png', dpi=300, bbox_inches='tight')
   plt.show()

def gr2():
   plt.figure(figsize=(10,6))
   plt.plot(arr['b'], arr['actual_error'], 'bo-', linewidth=2, markersize=8,
label='Actual error')
   plt.plot(arr['b'], arr['theoretical_error'], 'r--', linewidth=2,
label='Theoretical error (1.04/√M)')
   for i, (b, act, theo) in enumerate(zip(arr['b'], arr['actual_error'],
arr['theoretical_error'])):
       plt.annotate(f'{act:.1f}%', xy=(b, act), xytext=(0, 10), textcoords='offset
points', ha='center')
       plt.annotate(f'{theo:.1f}%', xy=(b, theo), xytext=(0, -15),
textcoords='offset points', ha='center', color='red')
   plt.xlabel('B (bits)', fontsize=12)
   plt.ylabel('Error (%)', fontsize=12)
   plt.title('HLL Error vs Number of Bits (B)', fontsize=14)
   plt.legend(loc='best')
   plt.grid(True, alpha=0.3)
```

```
    plt.xticks(arr['b'])
    plt.savefig('plot2_error_by_b.png', dpi=300, bbox_inches='tight')
    plt.show()

def gr3():
    plt.figure(figsize=(10,6))
    x = np.arange(len(arr['b']))
    width = 0.35
    plt.bar(x - width/2, arr['estimate'], width, label='HLL Estimate',
color='lightblue', edgecolor='black')
    plt.bar(x + width/2, [unq]*len(arr['b']), width, label='True Value (50000)',
color='lightgreen', edgecolor='black', alpha=0.7)
    for i, (est, err) in enumerate(zip(arr['estimate'], arr['actual_error'])):
        plt.text(i - width/2, est + 1000, f'{est:.0f}\n({err:.1f}%)', ha='center',
va='bottom', fontsize=9)
    plt.axhline(y=unq, color='red', linestyle='--', alpha=0.5)
    plt.xlabel('B (bits) / M (registers)', fontsize=12)
    plt.ylabel('Unique count', fontsize=12)
    plt.title('HLL Estimates for Different B Values', fontsize=14)
    plt.xticks(x, [f'B={b}\nM={m}' for b, m in zip(arr['b'], arr['M'])])
    plt.legend(loc='best')
    plt.grid(True, alpha=0.3, axis='y')
    plt.savefig('plot3_estimates_by_b.png', dpi=300, bbox_inches='tight')
    plt.show()


if __name__ == "__main__":
    gr1()
    gr3()
    gr3()
```